CW2: Active Learning on CIFAR-10
TypiClust Reproduction and Adaptive Extension

Arrnav Gupta
24032130
King's College London

March 26, 2026

Abstract

This report reproduces the low-budget TypiClust setting of Hacohen et al. [2] on CIFAR-10 using self-supervised image representations, then evaluates an adaptive modification that preserves pure typicality in the cold-start regime and gradually adds novelty and uncertainty later. The final pipeline used 500 SimCLR pretraining epochs on an RTX 3070 and compared Random, Entropy, TypiClust, and TypiClust+Adaptive over five active learning rounds with matched seeds.

1 Introduction

Active learning reduces annotation cost by querying only a small subset of examples for labeling. Hacohen et al. [2] argue that the low-budget regime is special: when only a few labels are available per round, representative examples can matter more than classical uncertainty heuristics. CIFAR-10 is a useful benchmark for testing that claim because it is widely used, computationally manageable, and still exposes whether a query strategy can cover the data distribution under severe label scarcity.

2 Methodology

TypiClust computes an embedding of the unlabeled pool, clusters the pool, and repeatedly selects the most typical point from the cluster with the fewest labeled examples. Typicality is measured as inverse average distance to nearest neighbours:

$$\text{Typicality}(x) = \left(\frac{1}{K} \sum_{x_i \in K\text{-NN}(x)} \|x - x_i\|_2 \right)^{-1}.$$

Diversity is therefore enforced through clustering, while representativeness is enforced within each cluster through local density. The reproduction uses CIFAR-10, a SimCLR-style ResNet-18 encoder [1] for self-supervised representation learning, L2-normalized embeddings, and a supervised ResNet-18 classifier retrained after each query round. Following Appendix F of the paper, clustering uses $K = \min(|L| + B, \texttt{max_clusters})$, typicality uses up to 20 neighbours, and tiny clusters are filtered before selection. The final automatic run used 500 SimCLR pretraining epochs on the RTX to build frozen embeddings for TypiClust and TypiClust+Adaptive, then retrained a supervised ResNet-18 classifier after every query round. The study evaluated 5 active learning rounds with matched seeds across Random, Entropy, TypiClust, and TypiClust+Adaptive.

3 Results

Table 1 summarises the paper's CIFAR-10 claims, while Table 2 reports the final-round numbers from the completed 500-epoch reproduction. This makes it possible to compare the published trend with the behaviour of the local run directly.

Table 1: Qualitative CIFAR-10 findings reported by Hacohen et al. [2].

Figure	Setting	Reported finding
Fig. 4a	Fully supervised, 5 low-budget iterations	Both TypiClust variants outperform Random and uncertainty baselines by a large margin.
Fig. 5a	Fully supervised with self-supervised embedding	TypiClust again outperforms the baselines by a large margin.
Fig. 6a	Semi-supervised FlexMatch with 10 CIFAR-10 labels	Both TypiClust variants outperform random sampling, while uncertainty baselines do not improve on Random.

The completed reproduction used 500 self-supervised pretraining epochs on the RTX. At round 5, the strategy ranking was Random (20.82%), TypiClust+Adaptive (20.77%), Entropy (19.93%), TypiClust (17.97%). A clustering-based strategy beat Random in rounds 1, 2, 3, 4, but Random finished highest at round 5. TypiClust+Adaptive changed the final-round mean by 2.80% relative to the original TypiClust.

Table 2: Final-round reproduction results on CIFAR-10.

Strategy	Round	Mean Acc.	SE
TypiClust	5	17.97%	0.54%
TypiClust+Adaptive	5	20.77%	1.06%
Random	5	20.82%	0.63%
Entropy	5	19.93%	0.43%

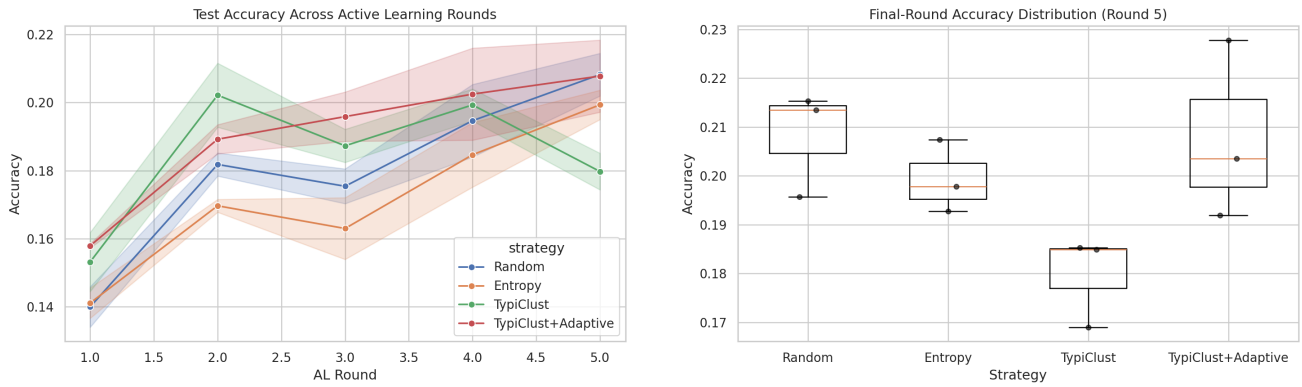

Figure 1: Left: mean test accuracy by active learning round. Right: final-round distribution across three repeats.

4 Statistical Analysis

Paired tests were computed on the final round over matched repeats ($n = 3$). Reported values include paired t-tests and Wilcoxon signed-rank tests, with significance interpreted at $\alpha = 0.05$. Because the repetition count is small, effect size and direction are discussed alongside p-values rather than relying on significance alone. Compared with TypiClust, TypiClust+Adaptive changed the final-round mean by 2.80% ($p = 0.109$ paired t-test; $p = 0.125$ Wilcoxon). Compared with Random, TypiClust+Adaptive changed the final-round mean by -0.04% ($p = 0.523$ paired t-test; $p = 0.625$ Wilcoxon). Compared with Random, TypiClust changed the final-round mean by -2.84% ($p = 0.944$ paired t-test; $p = 1.000$ Wilcoxon). None of the final-round comparisons crossed the 0.05 significance threshold, so the observed differences should be interpreted as directional rather than conclusive.

5 Algorithm Modification

The targeted improvement pass replaces the original novelty-only variant with a late-stage hybrid score. The modified method keeps pure TypiClust in the earliest low-budget rounds, activates a normalized novelty term once the labeled set has initial coverage, and then adds uncertainty only in later rounds when the classifier has enough supervision to make entropy informative:

$$\text{Score}(x) = \widehat{\text{Typicality}}(x) + \lambda_t \cdot \widehat{\text{Novelty}}(x) + \mu_t \cdot \widehat{\text{Entropy}}(x),$$

where λ_t increases from round 3, μ_t increases from round 4, and all terms are min-max normalized within the chosen cluster. This is justified by the paper's budget-dependent argument and by the tuning evidence from the reproduction: when the labeled pool is almost empty, representative points should dominate; after that, novelty improves coverage; only later should the method trust classifier uncertainty. The evaluation therefore compares TypiClust and TypiClust+Adaptive with identical seeds, budgets, cached SSL embeddings, and training settings.

6 Conclusion

A clustering-based method led Random throughout the pre-final rounds, but the final configuration did not reproduce the paper's strongest claim at round 5: Random remained the best final strategy. TypiClust+Adaptive improved TypiClust by 2.80% at the final round, remained narrowly below Random, and surpassed Entropy. The modification should therefore be presented as a modest improvement over the base method rather than a universal win.

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning (ICML)*, 2020.

[2] Guy Hacohen, Avihu Dekel, and Daphna Weinshall. Active learning on a budget: Opposite strategies suit high and low budgets. In *International Conference on Machine Learning (ICML)*, 2022.

Code Appendix (Page 3+)

This appendix accompanies the submission with notebook printouts of the key source files for the original and modified query strategies. GitHub repository URL: `https://github.com/ToastyPencil/CW2_ML`.

Notebook Printout: `typiclust_original.ipynb`

TypiClust Original

Notebook printout for the coursework appendix. Each code cell below contains the submitted implementation source.

What this notebook contains

- The submitted implementation of the original TypiClust acquisition rule.
- The exact source file used by the coursework pipeline.

How to use it

- Treat this notebook as a printout appendix for the report.
- The code cell is copied directly from the project source so markers can inspect the implementation without opening the repository.

```python
# File: src/cw2_ml/al/typiclust.py
from __future__ import annotations

from collections import defaultdict
from typing import Iterable

import numpy as np
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.neighbors import NearestNeighbors


def _cluster_points(
    embeddings: np.ndarray,
    cluster_count: int,
    random_state: int,
) -> np.ndarray:
    if cluster_count <= 50:
        clusterer = KMeans(n_clusters=cluster_count, n_init=10, random_state=random_state)
    else:
        clusterer = MiniBatchKMeans(
            n_clusters=cluster_count,
            n_init=10,
            batch_size=1024,
            random_state=random_state,
        )
    return clusterer.fit_predict(embeddings)


def _typicality_scores(cluster_embeddings: np.ndarray, knn_k: int) -> np.ndarray:
    if cluster_embeddings.shape[0] == 1:
        return np.array([np.inf], dtype=np.float64)

    k = min(max(knn_k, 1), cluster_embeddings.shape[0] - 1)
    neighbors = NearestNeighbors(n_neighbors=k + 1, metric="euclidean")
    neighbors.fit(cluster_embeddings)
    distances, _ = neighbors.kneighbors(cluster_embeddings)
    mean_dist = distances[:, 1:].mean(axis=1)
    return 1.0 / (mean_dist + 1e-12)


def _choose_cluster(
    cluster_to_indices: dict[int, list[int]],
    effective_labeled: set[int],
    selected_set: set[int],
    min_cluster_size: int,
) -> int | None:
    candidate_meta: list[tuple[int, int, int]] = []
    for cluster_id, members in cluster_to_indices.items():
        if len(members) < min_cluster_size:
            continue
        available = [idx for idx in members if idx not in effective_labeled and idx not in selected_set]
        if not available:
            continue
        labeled_count = sum(idx in effective_labeled or idx in selected_set for idx in members)
        candidate_meta.append((cluster_id, labeled_count, len(members)))

    if not candidate_meta:
```

```
        return None

    candidate_meta.sort(key=lambda item: (item[1], -item[2], item[0]))
    return candidate_meta[0][0]


def select_typiclust_indices(
    embeddings: np.ndarray,
    labeled_indices: Iterable[int],
    budget: int,
    max_clusters: int = 500,
    knn_k: int = 20,
    min_cluster_size: int = 5,
    random_state: int = 42,
) -> list[int]:
    """Select query indices following TypiClust (TPC-RP) rules.

    The method clusters the entire pool, repeatedly picks a cluster with the fewest
    labeled points (breaking ties by larger cluster size), then selects the most
    typical point in that cluster using inverse kNN distance.
    """
    if budget <= 0 or embeddings.size == 0:
        return []

    n = embeddings.shape[0]
    labeled = {int(i) for i in labeled_indices if 0 <= int(i) < n}
    unlabeled = [i for i in range(n) if i not in labeled]
    if not unlabeled:
        return []

    budget = min(budget, len(unlabeled))
    cluster_count = max(1, min(len(labeled) + budget, max_clusters, n))

    labels = _cluster_points(embeddings, cluster_count, random_state)
    cluster_to_indices: dict[int, list[int]] = defaultdict(list)
    for idx, cluster_id in enumerate(labels.tolist()):
        cluster_to_indices[cluster_id].append(idx)

    selected: list[int] = []
    selected_set: set[int] = set()
    effective_labeled = set(labeled)

    while len(selected) < budget:
        chosen_cluster = _choose_cluster(
            cluster_to_indices=cluster_to_indices,
            effective_labeled=effective_labeled,
            selected_set=selected_set,
            min_cluster_size=min_cluster_size,
        )

        if chosen_cluster is None:
            break

        members = cluster_to_indices[chosen_cluster]
        member_embeddings = embeddings[np.array(members)]
        member_scores = _typicality_scores(member_embeddings, knn_k=min(knn_k, len(members)))

        best_idx = None
        best_score = float("-inf")
        for local_pos, global_idx in enumerate(members):
            if global_idx in effective_labeled or global_idx in selected_set:
                continue
            score = float(member_scores[local_pos])
            if score > best_score or (score == best_score and (best_idx is None or global_idx < best_idx)):
                best_score = score
                best_idx = global_idx

        if best_idx is None:
            break

        selected.append(best_idx)
        selected_set.add(best_idx)
        effective_labeled.add(best_idx)

    if len(selected) < budget:
        remaining = [i for i in unlabeled if i not in selected_set]
        selected.extend(remaining[: budget - len(selected)])

    return selected
```

Notebook Printout: `typiclust_modified.ipynb`

TypiClust+Adaptive Modified

Notebook printout for the coursework appendix. Each code cell below contains the submitted implementation source.

What this notebook contains

- The submitted implementation of the adaptive TypiClust modification.
- The round-dependent novelty and uncertainty schedule used in the final coursework evaluation.

How to use it

- Treat this notebook as a printout appendix for the report.
- The code cell is copied directly from the project source so markers can inspect the late-stage hybrid modification in one place.

```python
# File: src/cw2_ml/al/modified.py
from __future__ import annotations

from collections import defaultdict
from typing import Iterable

import numpy as np

from .typiclust import (
    _choose_cluster,
    _cluster_points,
    _typicality_scores,
    select_typiclust_indices,
)


def _novelty_scores(
    embeddings: np.ndarray,
    candidate_indices: list[int],
    labeled_indices: set[int],
) -> np.ndarray:
    if not labeled_indices:
        return np.zeros(len(candidate_indices), dtype=np.float64)

    labeled_embeddings = embeddings[np.array(sorted(labeled_indices))]
    candidate_embeddings = embeddings[np.array(candidate_indices)]
    distances = np.linalg.norm(
        candidate_embeddings[:, None, :] - labeled_embeddings[None, :, :],
        axis=2,
    )
    return distances.min(axis=1)


def _adaptive_lambda(
    round_idx: int,
    total_rounds: int,
    novelty_max_weight: float,
    novelty_start_round: int,
) -> float:
    if novelty_max_weight <= 0.0 or round_idx < novelty_start_round:
        return 0.0

    denom = max(total_rounds - novelty_start_round, 1)
    progress = (round_idx - novelty_start_round + 1) / denom
    progress = float(min(max(progress, 0.0), 1.0))
    return float(novelty_max_weight * progress)


def _normalize(values: np.ndarray) -> np.ndarray:
    if values.size == 0:
        return values.astype(np.float64, copy=False)

    normalized = np.zeros_like(values, dtype=np.float64)
    finite_mask = np.isfinite(values)
    if finite_mask.any():
        finite_values = values[finite_mask]
        vmin = float(finite_values.min())
        vmax = float(finite_values.max())
        if abs(vmax - vmin) >= 1e-12:
            normalized[finite_mask] = (finite_values - vmin) / (vmax - vmin)
    normalized[np.isposinf(values)] = 1.0
    normalized[np.isneginf(values)] = 0.0
    normalized[np.isnan(values)] = 0.0
    return normalized
```

```python
def select_typiclust_adaptive_indices(
    embeddings: np.ndarray,
    labeled_indices: Iterable[int],
    budget: int,
    round_idx: int,
    total_rounds: int,
    novelty_max_weight: float = 0.2,
    novelty_start_round: int = 2,
    uncertainty_scores: dict[int, float] | None = None,
    uncertainty_max_weight: float = 0.0,
    uncertainty_start_round: int = 4,
    max_clusters: int = 500,
    knn_k: int = 20,
    min_cluster_size: int = 5,
    random_state: int = 42,
) -> list[int]:
    """TypiClust with a round-dependent novelty term against labeled points.

    ``round_idx`` is interpreted as the 1-based active learning round number.
    """
    if budget <= 0 or embeddings.size == 0:
        return []

    novelty_weight = _adaptive_lambda(
        round_idx=round_idx,
        total_rounds=total_rounds,
        novelty_max_weight=novelty_max_weight,
        novelty_start_round=novelty_start_round,
    )
    uncertainty_weight = _adaptive_lambda(
        round_idx=round_idx,
        total_rounds=total_rounds,
        novelty_max_weight=uncertainty_max_weight,
        novelty_start_round=uncertainty_start_round,
    )
    if novelty_weight <= 0.0 and uncertainty_weight <= 0.0:
        return select_typiclust_indices(
            embeddings=embeddings,
            labeled_indices=labeled_indices,
            budget=budget,
            max_clusters=max_clusters,
            knn_k=knn_k,
            min_cluster_size=min_cluster_size,
            random_state=random_state,
        )

    n = embeddings.shape[0]
    labeled = {int(i) for i in labeled_indices if 0 <= int(i) < n}
    unlabeled = [i for i in range(n) if i not in labeled]
    if not unlabeled:
        return []

    budget = min(budget, len(unlabeled))
    cluster_count = max(1, min(len(labeled) + budget, max_clusters, n))

    labels = _cluster_points(embeddings, cluster_count, random_state)
    cluster_to_indices: dict[int, list[int]] = defaultdict(list)
    for idx, cluster_id in enumerate(labels.tolist()):
        cluster_to_indices[cluster_id].append(idx)

    selected: list[int] = []
    selected_set: set[int] = set()
    effective_labeled = set(labeled)

    while len(selected) < budget:
        chosen_cluster = _choose_cluster(
            cluster_to_indices=cluster_to_indices,
            effective_labeled=effective_labeled,
            selected_set=selected_set,
            min_cluster_size=min_cluster_size,
        )
        if chosen_cluster is None:
            break

        members = cluster_to_indices[chosen_cluster]
        member_embeddings = embeddings[np.array(members)]
        typicality = _typicality_scores(member_embeddings, knn_k=min(knn_k, len(members)))

        available = [idx for idx in members if idx not in effective_labeled and idx not in selected_set]
        if not available:
            break

        member_positions = {idx: pos for pos, idx in enumerate(members)}
        available_positions = [member_positions[idx] for idx in available]
        available_typ = _normalize(np.array([typicality[pos] for pos in available_positions], dtype=np.float64))
        available_nov = _normalize(_novelty_scores(embeddings, available, effective_labeled))
        available_unc = _normalize(
            np.array(
                [float(uncertainty_scores.get(idx, 0.0)) if uncertainty_scores is not None else 0.0 for idx in available],
                dtype=np.float64,
            )
```

```
    )

    best_idx = None
    best_score = float("-inf")
    for local_pos, global_idx in enumerate(available):
        score = float(
            available_typ[local_pos]
            + novelty_weight * available_nov[local_pos]
            + uncertainty_weight * available_unc[local_pos]
        )
        if score > best_score or (score == best_score and (best_idx is None or global_idx < best_idx)):
            best_score = score
            best_idx = global_idx

    if best_idx is None:
        break

    selected.append(best_idx)
    selected_set.add(best_idx)
    effective_labeled.add(best_idx)

if len(selected) < budget:
    remaining = [i for i in unlabeled if i not in selected_set]
    selected.extend(remaining[: budget - len(selected)])

return selected
```